

Aula 05 – Projeto integrador

Requisitos hoje; implementação, qualidade e deploy na próxima aula

Prof. Lucca Abbado Neres

Centro Universitário Fundação Assis Gurgacz – FAG

2026



A proposta desta aula

Hoje a disciplina muda de formato: em vez de aprender uma ferramenta nova, a turma vai **integrar o que já aprendeu** em um projeto pequeno, mas tratado como projeto real.

Hoje

Planejar o sistema e produzir a primeira versão do documento de requisitos.

Na próxima aula

Implementar, testar, proteger o repositório, realizar deploy e validar com um cliente real.

Formação das equipes

Backend

API, regras de negócio, integração com banco e persistência.

Frontend

Interface, fluxo do usuário, formulários e consumo da API.

Product Owner

Escopo, prioridades, histórias de usuário e decisões de produto.

Qualidade

Critérios de aceite, testes, revisão e evidências.

Grupos de 4 a 5 pessoas

Os quatro papéis acima são obrigatórios. Em grupos com 5 integrantes, a quinta pessoa pode assumir **DevOps/infraestrutura** ou atuar como apoio full-stack, sem eliminar nenhum dos papéis principais.

O problema deve ser pequeno e utilizável

O objetivo não é construir uma plataforma gigantesca. O sistema precisa ser simples o bastante para ficar funcional, mas completo o suficiente para exigir trabalho de equipe.

Bom escopo

- ▶ agenda simples;
- ▶ catálogo com cadastro;
- ▶ controle de reservas;
- ▶ pedidos de uma pequena loja;
- ▶ chamados ou solicitações;
- ▶ sistema para um problema real conhecido pelo grupo.

Evitar

- ▶ marketplace completo;
- ▶ rede social;
- ▶ ERP;
- ▶ sistema com dezenas de telas;
- ▶ escopo que dependa de recursos não estudados;
- ▶ “vamos decidir enquanto programamos”.

Entregável da aula de hoje

Ao final da aula, cada grupo deve possuir uma **primeira versão do documento de requisitos** contendo:

- ▶ visão e objetivo do sistema;
- ▶ requisitos funcionais;
- ▶ histórias de usuário;
- ▶ casos de uso;
- ▶ regras de negócio;
- ▶ requisitos não funcionais;
- ▶ escopo mínimo da versão que será implementada;
- ▶ divisão inicial das responsabilidades da equipe.

Resultado esperado

A próxima aula deve começar com o grupo sabendo **o que construir, para quem, quais regras respeitar e como saber se está correto.**

1. Comecem pelo problema

Antes de escrever RF01, RF02 e RF03, respondam quatro perguntas.

Quem é o
usuário?

Qual problema
ele possui?

O que o sistema
resolve?

Como saberemos
que funcionou?

Regra prática

Se o grupo não consegue explicar o sistema em duas ou três frases, o escopo ainda não está claro.

2. Definam o MVP da próxima aula

MVP aqui significa a menor versão que demonstra valor e pode ser usada por alguém fora da equipe.

Essencial

Funcionalidades sem as quais o sistema não cumpre seu propósito.

Depois

Melhorias, filtros sofisticados, dashboards, temas, notificações e outras extensões.

Pergunta que o PO deve fazer: “Se só pudermos entregar três ou quatro funções, quais tornam o sistema realmente utilizável?”

3. Requisitos funcionais

Requisitos funcionais descrevem **o que o sistema deve fazer**.

Formato simples

RF01 – Cadastrar solicitação. O sistema deve permitir que um usuário cadastre uma solicitação informando título, descrição e categoria.

- ▶ use identificador único: RF01, RF02, RF03...;
- ▶ escreva uma função verificável por requisito;
- ▶ evite termos vagos como “ser completo”, “ser moderno” ou “ser intuitivo”;
- ▶ escreva pensando em algo que depois possa ser testado.

4. Histórias de usuário

Histórias de usuário descrevem uma necessidade a partir do ponto de vista de quem usa o produto.

Estrutura

Como cliente, **quero** acompanhar o status do meu pedido, **para** saber se ele já pode ser retirado.

- ▶ “Como” = papel ou tipo de usuário;
- ▶ “quero” = objetivo;
- ▶ “para” = valor ou motivo;
- ▶ uma história pode se relacionar com um ou mais requisitos funcionais.

História boa precisa de critério de aceite

A história diz o que o usuário deseja. O critério de aceite ajuda a decidir quando aquela história está pronta.

História

Como cliente, quero cadastrar uma reserva para garantir meu horário.

Critérios de aceite

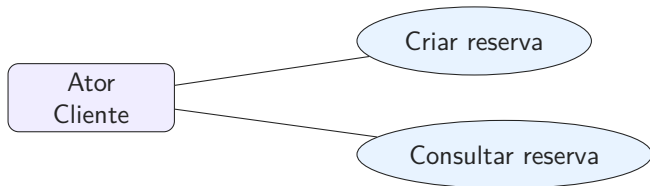
- ▶ não aceitar horário no passado;
- ▶ exigir nome e contato;
- ▶ impedir duas reservas para o mesmo horário;
- ▶ confirmar o cadastro ao usuário.

Importância para DevOps

Critérios de aceite dão origem a testes. Requisito, implementação e validação devem conversar entre si.

5. Casos de uso

Casos de uso organizam a interação entre um ator e o sistema para alcançar um objetivo.



- ▶ identifique os atores principais;
- ▶ selecione os casos de uso do MVP;
- ▶ descreva fluxo principal e exceções relevantes;
- ▶ não é necessário modelar dezenas de casos.

Caso de uso em texto: nível suficiente para hoje

Caso de uso	Cadastrar reserva
Ator principal	Cliente
Pré-condição	O horário deve estar disponível.
Fluxo principal	Usuário escolhe data e horário, informa dados, confirma e recebe retorno do sistema.
Fluxo alternativo	Horário ocupado: o sistema não conclui a reserva e solicita outra opção.
Pós-condição	Reserva persistida no banco e disponível para consulta.

Evitem excesso de burocracia

O caso de uso deve esclarecer a implementação e os testes, não virar um formulário enorme sem valor para o projeto.

6. Regras de negócio

Regras de negócio são restrições ou políticas do domínio que o sistema precisa respeitar.

Exemplos

RN01. Uma reserva só pode ser feita com pelo menos 2 horas de antecedência.

RN02. Um horário não pode possuir duas reservas ativas.

RN03. Apenas o responsável pode cancelar uma reserva confirmada.

Não confundir

“O sistema deve possuir botão verde” não é regra de negócio. “Uma reserva cancelada não libera reembolso após o prazo X” pode ser.

7. Requisitos não funcionais

Requisitos não funcionais descrevem qualidades e restrições do sistema.

- ▶ desempenho;
- ▶ segurança;
- ▶ disponibilidade;
- ▶ usabilidade;
- ▶ compatibilidade;
- ▶ manutenibilidade.

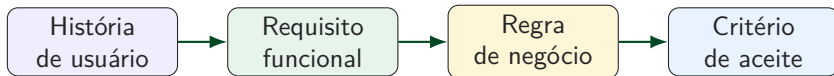
Exemplo testável

RNF01. As telas principais devem ser utilizáveis em navegador desktop e celular sem perda das funções do MVP.

Prefiram requisitos que possam ser **demonstrados, medidos ou verificados**.

8. Ligando os artefatos

Os itens do documento não devem existir isoladamente.



Ideia central

Uma necessidade do usuário gera comportamento do sistema; esse comportamento respeita regras; e o grupo define como verificar se a entrega está correta.

Uma tabela de rastreabilidade já ajuda muito

História	Requisito	Regra associada	Teste futuro
HU01	RF01	RN01, RN02	integração: criar reserva válida sistema: impedir horário duplicado unidade: validar cancelamento
HU01	RF01	RN02	
HU02	RF03	RN03	

Por que isso é útil?

Na próxima aula, essa tabela evita “testar qualquer coisa”. A equipe já sabe quais comportamentos críticos precisam ser comprovados.

Na próxima aula: fluxo de desenvolvimento

A equipe não deve programar diretamente na branch principal.



Regra da atividade

A main deve estar protegida contra merge sem aprovação. O fluxo precisa passar por Pull Request.

Proteção da branch principal

O repositório deverá impedir que a equipe ignore o processo de revisão.

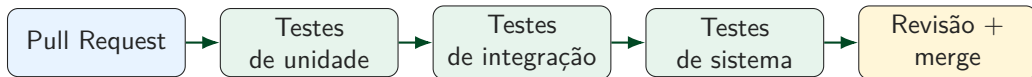
- ▶ exigir Pull Request antes do merge;
- ▶ exigir pelo menos uma aprovação;
- ▶ exigir que os checks definidos pela equipe estejam concluídos com sucesso;
- ▶ evitar push direto na `main`;
- ▶ resolver conversas de revisão antes do merge, quando configurado.

Objetivo

A regra não existe para “atrapalhar o merge”. Ela transforma revisão e qualidade em parte obrigatória do processo de entrega.

Integração contínua: o mínimo esperado

Cada Pull Request deve disparar automaticamente a verificação do projeto.



Importante

Não basta possuir testes no repositório. Eles precisam participar do fluxo de CI e produzir evidência automática de aprovação ou falha.

Três níveis de teste no projeto

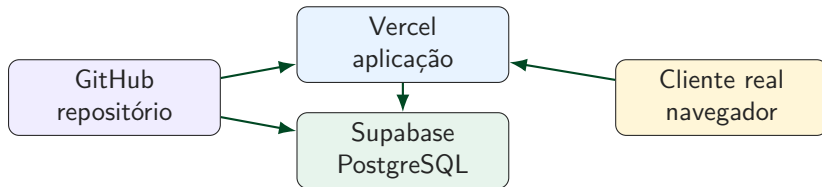
Tipo	O que isolar/validar	Exemplo
Unidade	função, regra ou componente pequeno	validar cálculo, formato, regra de status
Integração	comunicação entre partes reais	API + banco, serviço + repositório
Sistema	comportamento do produto como usuário	abrir aplicação, cadastrar, consultar, editar ou excluir

Não é competição por quantidade

Três testes úteis e bem escolhidos valem mais que dezenas de testes que não protegem nenhum comportamento importante.

Arquitetura de deploy da atividade

O sistema final será publicado usando Vercel para a aplicação e Supabase para o banco de dados.



O foco

O projeto deve sair da máquina dos desenvolvedores e se tornar acessível para outra pessoa testar.

Deploy da aplicação: nível de abstração

Não é necessário decorar um tutorial. O processo esperado é compreender o encadeamento.

1. manter o projeto versionado no GitHub;
2. conectar o repositório ao projeto da Vercel;
3. configurar comandos de build e variáveis necessárias;
4. publicar a branch de produção;
5. verificar logs e corrigir falhas de build;
6. confirmar que o endereço público acessa corretamente o sistema.

Boa prática

O deploy deve depender do repositório, não de arquivos enviados manualmente de uma máquina específica.

Deploy do banco: o mínimo obrigatório

O banco remoto deve possuir o esquema necessário para o sistema funcionar no Supabase.

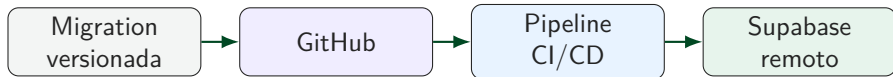
- ▶ criar o projeto remoto;
- ▶ definir tabelas, relacionamentos, restrições e dados mínimos necessários;
- ▶ configurar a aplicação para acessar o ambiente remoto;
- ▶ documentar como o esquema foi criado ou atualizado;
- ▶ validar que o sistema publicado realmente persiste e consulta dados.

Não entreguem um “frontend de mentira”

Se o sistema promete persistência, as operações principais devem utilizar o banco remoto de verdade.

Adicional de nota: banco como código

O adicional é automatizar a evolução do esquema do banco com **Supabase CLI + migrações versionadas**.



Ideia

Mudanças de banco deixam de ser cliques manuais no painel e passam a existir como arquivos no repositório, aplicados de forma reproduzível ao ambiente remoto.

Condição do bônus

Automação não compensa sistema quebrado. O adicional só faz sentido se o projeto apresentar qualidade adequada.

O cliente real faz parte da avaliação

Depois do deploy, o sistema precisa ser usado por alguém que **não participou do desenvolvimento**.

- ▶ escolher uma pessoa compatível com o público-alvo;
- ▶ pedir que ela execute a principal tarefa do sistema;
- ▶ evitar orientar cada clique durante o teste;
- ▶ observar dificuldades, erros e dúvidas;
- ▶ registrar o uso em vídeo;
- ▶ coletar uma impressão final breve do cliente.

Por que fazer isso?

Código aprovado pela equipe não significa automaticamente produto compreensível para quem vai utilizá-lo.

Critérios mínimos da entrega final

A versão da próxima aula deve demonstrar:

- ▶ sistema coerente com o documento de requisitos;
- ▶ funcionalidades essenciais do MVP operacionais;
- ▶ persistência em Supabase;
- ▶ deploy acessível publicamente;
- ▶ testes de unidade, integração e sistema;
- ▶ CI executando verificações automaticamente;
- ▶ ambiente de desenvolvimento reproduzível e documentado (Docker/Compose, quando aplicável);
- ▶ branch principal protegida e merge por PR aprovado;
- ▶ vídeo com cliente real utilizando o sistema.

Roteiro de trabalho para HOJE

1. formar grupos e definir papéis;
2. escolher problema e público-alvo;
3. definir o MVP;
4. escrever requisitos funcionais;
5. criar histórias e critérios de aceite;
6. elaborar casos de uso principais;
7. registrar regras de negócio;
8. definir requisitos não funcionais;
9. revisar o documento em grupo;
10. planejar as primeiras tarefas da implementação.

Referências técnicas para a próxima aula

As ferramentas abaixo possuem documentação oficial suficiente para a implementação. A aula não pretende substituir essa documentação com um tutorial passo a passo.

- ▶ GitHub Docs – branch protection, rulesets, pull requests e required status checks;
- ▶ Vercel Docs – deploy de repositórios Git e deploys automáticos;
- ▶ Supabase Docs – CLI, migrations, `supabase link`, `supabase db push` e CI/CD;
- ▶ documentação oficial das bibliotecas escolhidas pelo grupo para testes.

Competência esperada

Parte do trabalho profissional é saber localizar a documentação correta e transformar requisitos em uma solução reproduzível.

Organizem a equipe.

Definam o problema.

Fechem o MVP.

Escrevam requisitos que possam ser testados.

Meta da aula

Sair com uma especificação boa o suficiente para que a implementação da próxima semana comece imediatamente.